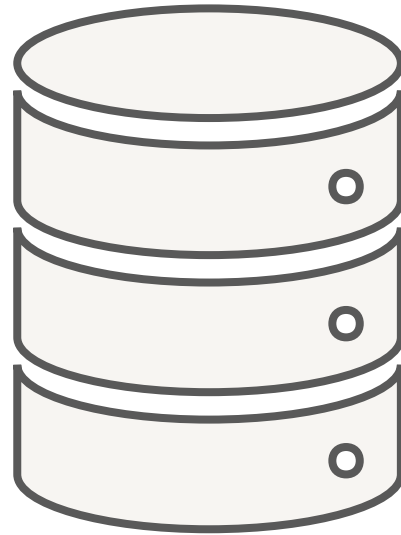

Interagir avec la base de données

Introduction

Presque toutes les applications web modernes interagissent avec une base de données. Laravel rend l'interaction avec les bases de données extrêmement simple à travers une variété de bases de données supportées en utilisant le SQL brut, un constructeur de requêtes fluide et l'ORM Eloquent. Actuellement, Laravel fournit un support de première partie pour cinq bases de données :

- MariaDB 10.3+
- MySQL 5.7+
- PostgreSQL 10.0+
- SQLite 3.8.8+
- SQL Server 2017+

Configuration de la base de données



Configuration

La configuration des services de base de données de Laravel se trouve dans le fichier de configuration **config/database.php** de votre application.

Dans ce fichier, vous pouvez définir toutes vos connexions à la base de données, ainsi que spécifier quelle connexion doit être utilisée par défaut. La plupart des options de configuration de ce fichier sont déterminées par les valeurs des variables d'environnement de votre application.

Configuration de SQLite

Les bases de données **SQLite** sont contenues dans un seul fichier sur votre système de fichiers.

Après la création de la base de données, vous pouvez facilement configurer vos variables d'environnement pour qu'elles pointent vers cette base de données en plaçant le chemin absolu de la base de données dans la variable d'environnement **DB_DATABASE** :

```
DB_CONNECTION=sqlite
```

```
DB_DATABASE=/absolute/path/to/database.sqlite
```

Configuration de MS SQL Server

Pour utiliser une base de données Microsoft SQL Server, vous devez vous assurer que les extensions PHP **sqlsrv** et **pdo_sqlsrv** sont installées, ainsi que toutes les dépendances qu'elles peuvent nécessiter, comme le pilote **ODBC Microsoft SQL**.

Configuration à l'aide d'URLs

Généralement, les connexions aux bases de données sont configurées à l'aide de plusieurs valeurs de configuration telles que **host**, **database**, **username**, **password**, etc. Chacune de ces valeurs de configuration a sa propre variable d'environnement correspondante. Cela signifie que lorsque vous configurez vos informations de connexion à la base de données sur un serveur de production, vous devez gérer plusieurs variables d'environnement.

Configuration à l'aide d'URLs

Certains fournisseurs de bases de données gérées, comme AWS et Heroku, fournissent une "URL" de base de données unique qui contient toutes les informations de connexion à la base de données dans une seule chaîne. Un exemple d'URL de base de données peut ressembler à ce qui suit :

```
mysql://root:password@127.0.0.1/forge?charset=UTF-8
```

Ces URL suivent généralement une convention de schéma standard :

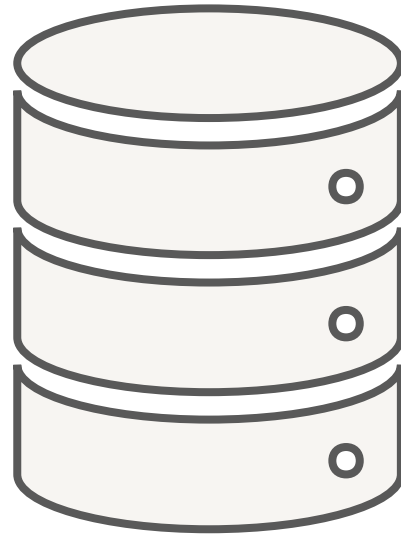
```
driver://username:password@host:port/database?options
```


Configuration à l'aide d'URLs

Pour des raisons de commodité, Laravel prend en charge ces URL comme une alternative à la configuration de votre base de données avec de multiples options de configuration.

Si l'option de configuration url (ou la variable d'environnement **DATABASE_URL** correspondante) est présente, elle sera utilisée pour extraire les informations de connexion et d'identification de la base de données.

Connexions



Connexions de lecture et d'écriture

Il peut arriver que vous souhaitiez utiliser une connexion à la base de données pour les instructions **SELECT**, et une autre pour les instructions **INSERT**, **UPDATE** et **DELETE**. Avec Laravel, c'est un jeu d'enfant, et les connexions appropriées seront toujours utilisées, que vous utilisiez des requêtes brutes, le générateur de requêtes ou l'ORM Eloquent.

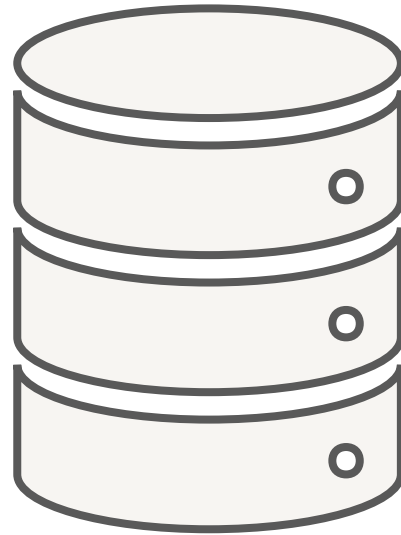
```
'mysql' => [  
  'read' => [  
    'host' => [  
      '192.168.1.1',  
      '196.168.1.2',  
    ],  
  ],  
  'write' => [  
    'host' => [  
      '196.168.1.3',  
    ],  
  ],  
  'sticky' => true,  
  'driver' => 'mysql',  
  'database' => 'database',  
  'username' => 'root',  
  'password' => "",  
  'charset' => 'utf8mb4',  
  'collation' => 'utf8mb4_unicode_ci',  
  'prefix' => "",  
],
```

Connexions de lecture et d'écriture

Les clés **read** et **write** ont des valeurs de tableau contenant une seule clé : **host**.

Le reste des options de base de données pour les connexions en lecture et en écriture seront fusionnées à partir du tableau de configuration principal de **mysql**.

Exécution de requêtes SQL



La requête **SELECT**

Pour exécuter une requête **SELECT** de base, vous pouvez utiliser la méthode **select** de la façade **DB** :

Le premier argument transmis à la méthode **select** est la requête SQL, tandis que le deuxième argument est tout paramètre qui doit être lié à la requête.

```
public function index()
{
    $users = DB::select('select * from users where active = ?', [1]);

    return view('user.index', ['users' => $users]);
}
```

La requête SELECT

La méthode **select** retournera toujours un tableau de résultats. Chaque résultat du tableau sera un **objet PHP stdClass** représentant un enregistrement de la base de données :

```
$users = DB::select('select * from users');  
  
foreach ($users as $user) {  
    echo $user->name;  
}
```


Sélection de valeurs scalaires

Parfois, votre requête de base de données peut aboutir à une seule valeur scalaire. Au lieu de devoir récupérer le résultat scalaire de la requête à partir d'un objet d'enregistrement, Laravel vous permet de récupérer cette valeur directement en utilisant la méthode scalaire :

```
$burgers = DB::scalar(  
    "select count(case when food = 'burger' then 1 end) as burgers from menu"  
);
```

Utilisation de liaisons nommées

Au lieu d'utiliser ? pour représenter vos liaisons de paramètres, vous pouvez exécuter une requête en utilisant des liaisons nommées :

```
$results = DB::select('select * from users where id = :id', ['id' => 1]);
```

L'instruction INSERT

Pour exécuter une instruction d'insertion, vous pouvez utiliser la méthode **insert** de la façade **DB**. Comme **select**, cette méthode accepte la requête SQL comme premier argument et les liaisons comme second argument :

```
use Illuminate\Support\Facades\DB;
```

```
DB::insert('insert into users (id, name) values (?, ?)', [1, 'Marc']);
```

L'instruction UPDATE

La méthode **update** doit être utilisée pour mettre à jour les enregistrements existants dans la base de données. Le nombre de lignes affectées par l'instruction est renvoyé par la méthode :

```
use Illuminate\Support\Facades\DB;

$affected = DB::update(
    'update users set votes = 100 where name = ?',
    ['Anita']
);
```

L'instruction DELETE

La méthode **delete** doit être utilisée pour supprimer des enregistrements de la base de données. Comme pour la méthode **update**, le nombre de lignes affectées sera renvoyé par la méthode :

```
use Illuminate\Support\Facades\DB;  
  
$deleted = DB::delete('delete from users');
```

Les instructions de base

Certaines instructions de base de données ne renvoient aucune valeur. Pour ces types d'opérations, vous pouvez utiliser la méthode **statement** de la façade **DB** :

```
use Illuminate\Support\Facades\DB;
```

```
DB::statement('drop table users');
```

Utilisation de connexions multiples

Si votre application définit plusieurs connexions dans votre fichier de configuration **config/database.php**, vous pouvez accéder à chaque connexion via la méthode de **connexion** fournie par la façade **DB**. Le nom de la connexion transmis à la méthode de connexion doit correspondre à l'une des connexions répertoriées dans votre fichier de configuration **config/database.php** ou configurées au moment de l'exécution à l'aide de la méthode **config** :

```
use Illuminate\Support\Facades\DB;
```

```
$users = DB::connection('sqlite')->select('/* ... */');
```